Homework 3: Typical CNNs

MNIST Classification with PyTorch

Total: 5 points

Answer the written questions and implement **at least two CNN architectures** for MNIST classification using PyTorch. Submit a **PDF report and all source code**. You may write your answers and report in **English or Chinese**.

No starter notebook or code is provided. Write all code yourself; PyTorch building blocks, TorchVision datasets and transforms, and standard data-loading and plotting libraries are allowed.

1. Typical CNNs: written questions (2 points)

Answer the following four questions in your PDF report. Each question is worth **0.5 point**. Explain your reasoning clearly.

1.1. AlexNet. Why can using ReLU instead of a saturating sigmoid activation make a CNN such as AlexNet easier to train?

1.2. VGG. Why does VGG favor stacking small 3×3 convolutional layers rather than using a single larger convolutional kernel?

1.3. GoogLeNet. How do 1×1 convolutions in an Inception module help control computational cost?

1.4. ResNet. How does the residual connection $y = F(x) + x$ help optimize a deep network?

2. MNIST classification with PyTorch (3 points)

Use PyTorch to implement and train **at least two structurally different CNNs** for MNIST classification. Include the following in your PDF report:

- **Model architectures:** show the structure of each CNN using a diagram or a layer table.
- **Dataset and splits:** describe MNIST, preprocessing, and the training/validation/test split. Create the validation set from the official training split; reserve the official test set for final evaluation.
- **Training hyperparameters:** state the loss, optimizer, learning rate, batch size, number of epochs and random seed for each model.
- **Learning curves:** plot training and validation loss curves and accuracy curves for each model.
- **Test results:** present both models' test loss and accuracy in a comparison table.
- **Bad cases:** display misclassified test images with their true and predicted labels, and analyze possible reasons for the errors.

Report and submission

Submit a **PDF report (no page limit)** with your name/student ID and all answers and results, together with **all source code**, dependencies and run instructions. Python scripts or self-authored notebooks are accepted. Work independently, report actual results and cite external material. Do not submit dataset files or environment directories.